PRÁCTICA 1 · RA1

Laboratorio multilenguaje para IA

Un mismo problema, cuatro ecosistemas tecnológicos

M3 · Programació d'Intel·ligència Artificial

RA1 · Lenguajes, herramientas y ecosistemas para IA

Práctica individual · 6–8 horas

Curso de Especialización en Inteligencia Artificial y Big Data

Índice

Contexto	3
Objetivos de aprendizaje	3
Condiciones comunes del experimento	3
Fase 1: Python	3
Fase 2: R	4
Fase 3: Julia	4
Fase 4: Java con Weka	4
Comparación tecnológica	4
Decisión razonada	5
Entrega	5
Criterios de evaluación	5
Consideraciones finales	5

1. Contexto

Elegir un lenguaje para un proyecto de inteligencia artificial no consiste únicamente en comparar su sintaxis. También intervienen el ecosistema de bibliotecas, el entorno de trabajo, la interoperabilidad, el rendimiento, la documentación, la comunidad y la facilidad de despliegue.

En esta práctica utilizarás un problema común y deliberadamente pequeño para observar esas diferencias. Trabajarás con el dataset Iris y con el algoritmo K-Nearest Neighbors (KNN) en cuatro ecosistemas: Python, R, Julia y Java.

> Reto
> El objetivo no es obtener la mayor precisión posible. KNN actúa como una carga de trabajo común para poder comparar, con evidencias, cómo se programa y se ejecuta una misma solución en cuatro lenguajes diferentes.

2. Objetivos de aprendizaje

Al finalizar la práctica deberás ser capaz de:

1. identificar la estructura de un programa de análisis y clasificación;
2. caracterizar lenguajes habituales en proyectos de IA;
3. seleccionar bibliotecas y entornos adecuados para una tarea concreta;
4. comparar sintaxis, tipado, gestión de dependencias e interoperabilidad;
5. obtener evidencias básicas de ejecución y reproducibilidad;
6. justificar qué tecnología utilizarías en distintos escenarios profesionales.

> Importante
> La evaluación se centra en la comparación tecnológica y en la justificación. No se valorará la optimización avanzada del modelo, que se trabajará en el RA2.

3. Condiciones comunes del experimento

Para que la comparación sea válida, las cuatro implementaciones deben respetar las mismas condiciones:

- dataset Iris completo, con cuatro variables de entrada y la especie como clase;
- partición estratificada del 80% para entrenamiento y 20% para prueba;
- semilla aleatoria equivalente a 42, siempre que la herramienta lo permita;
- normalización o estandarización de las variables antes de aplicar KNN;
- valor inicial k=5;
- métricas mínimas: accuracy y matriz de confusión;
- una visualización que ayude a comprender los datos o los resultados;
- registro de versiones del lenguaje, bibliotecas y entorno utilizados.

Si una biblioteca obliga a resolver alguna condición de forma diferente, deberás explicarlo en el informe.

4. Fase 1: Python

Utiliza Python en Google Colab, Jupyter o un entorno local reproducible.

Herramientas orientativas: `pandas`, `scikit-learn`, `matplotlib` y `seaborn`.

Debes:

1. cargar el dataset y mostrar su estructura;
2. describir los tipos de datos y la representación de la clase;
3. preparar la partición y el escalado;
4. entrenar y evaluar KNN;
5. generar una visualización;
6. identificar qué partes del código dependen de bibliotecas externas.

5. Fase 2: R

Utiliza R en Posit Cloud, RStudio o un entorno local reproducible.

Herramientas orientativas: `tidyverse`, `ggplot2`, `class` y paquetes de evaluación equivalentes.

Repite el flujo común y documenta especialmente:

- cómo representa R los factores y las tablas de datos;
- qué diferencias encuentras respecto a Python;
- cómo se instalan y cargan los paquetes;
- qué herramientas facilitan el análisis estadístico y la visualización.

6. Fase 3: Julia

Utiliza Julia en Pluto, Jupyter, VS Code o un entorno equivalente. Incluye en la entrega las instrucciones necesarias para reproducir el entorno.

Herramientas orientativas: `DataFrames.jl`, `MLJ.jl` y una biblioteca de visualización compatible.

Repite el flujo común y analiza:

- sintaxis, tipado y organización del código;
- gestión de paquetes y tiempos de primera ejecución;
- adecuación de Julia para computación científica;
- madurez y documentación del ecosistema utilizado.

7. Fase 4: Java con Weka

Implementa la solución en Java utilizando la API de Weka. Puedes emplear Maven o Gradle para gestionar la dependencia.

Debes utilizar `IBk` y documentar:

- carga del dataset en formato ARFF o conversión justificada;
- definición del atributo de clase;
- preparación, entrenamiento y evaluación;
- estructura de clases y cantidad de código necesaria;
- ventajas y limitaciones de la JVM para una aplicación de IA.

> Importante
> No es suficiente ejecutar Weka desde la interfaz gráfica. La práctica compara lenguajes, por lo que debe existir una implementación reproducible mediante código Java.

8. Comparación tecnológica

Completa una tabla comparativa basada en tus propias evidencias:

Criterio	Python	R	Julia	Java
Sintaxis y legibilidad				
Tipado y detección de errores				
Gestión de dependencias				
Bibliotecas de IA				
Visualización				
Interoperabilidad				
Rendimiento observado				
Facilidad de despliegue				
Documentación y comunidad				

Incluye, además, una tabla breve con las versiones, el tiempo de ejecución medido y la accuracy obtenida. No presentes esos valores como un benchmark científico: el dataset es pequeño y los entornos pueden ser diferentes.

9. Decisión razonada

Responde de forma argumentada:

1. ¿Qué lenguaje escogerías para crear rápidamente un prototipo de IA?
2. ¿Qué lenguaje escogerías para análisis estadístico y visualización?
3. ¿Cuál considerarías para cómputo científico intensivo?
4. ¿Cuál encajaría mejor en una organización basada en JVM?
5. ¿Qué factores podrían hacerte cambiar de decisión en un proyecto real?

No existe una respuesta universalmente correcta. Se valorará que relaciones cada elección con requisitos concretos.

10. Entrega

> Entrega
> Entrega un único archivo comprimido con el informe y las cuatro implementaciones. El trabajo debe poder revisarse sin reconstruir manualmente el entorno.

El paquete contendrá:

- informe en PDF de 5–7 páginas, sin contar anexos;
- una carpeta por lenguaje con código o notebook ejecutable;
- archivo `README` con instrucciones de ejecución;
- archivo de dependencias o versiones cuando corresponda;
- capturas o salidas que acrediten la ejecución;
- bibliografía con al menos cuatro fuentes, dos de ellas documentación oficial.

11. Criterios de evaluación

Criterio	Peso	Aspectos valorados
Implementaciones reproducibles	30%	Flujo completo en los cuatro lenguajes, condiciones comunes, organización y ejecución.
Caracterización de lenguajes y herramientas	25%	Sintaxis, tipado, bibliotecas, entornos, dependencias e interoperabilidad.
Comparación tecnológica	25%	Evidencias, identificación de ventajas y limitaciones y calidad de la tabla comparativa.
Rigor técnico y trazabilidad	10%	Versiones, fuentes, resultados, README y claridad de las evidencias.
Conclusiones y calidad del informe	10%	Decisión argumentada, estructura, legibilidad y capacidad de síntesis.
Total	100%	

12. Consideraciones finales

- Se valorará más una comparación bien razonada que pequeñas diferencias de accuracy.
- Todo fragmento de código externo debe citar su fuente.
- El uso de IA generativa debe respetar las normas indicadas por el profesorado.
- El alumnado deberá poder explicar y defender todo el código y las conclusiones entregadas.